K.R. MANGALAM UNIVERSITY

THE COMPLETE WORLD OF EDUCATION

School Of Engineering and Technology

Fundamentals of Algorithm Design and Analysis

Lab Assignment 1

(ENBC254)

(2025-26)

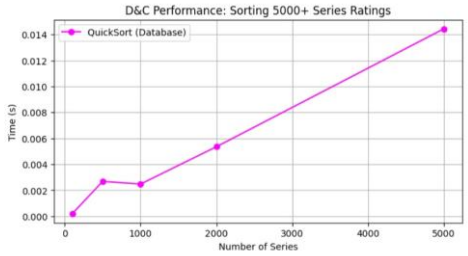
Name: Muskan
Roll Number: 2401840002
B.Sc. Data Science

Submitted to:
Mr. Shahjad
Assistant Professor

PROBLEM 1 and 2:

```
#1 and 2 DIVIDE AND CONQUER
#Complexity:  Average O(n log n)
Import time
Import random
Import numpy as np
Import matplotlib.pyplot as plt
Import sys

Sys.setrecursionlimit(3000)
Def sort_manhwa_ratings(arr):
    If len(arr) <= 1:
        Return arr
    Pivot = arr[len(arr) // 2]
    Left = [x for x in arr if x > pivot]
    Middle = [x for x in arr if x == pivot]
    Right = [x for x in arr if x < pivot]
    Return sort_manhwa_ratings(left) + middle + sort_manhwa_ratings(right)

Def benchmark_sorting():
    Sizes = [100, 500, 1000, 2000, 5000]
    Times = []
    For s in sizes:
        Data = [random.uniform(1.0, 10.0) for _ in range(s)]
        Start = time.time()
        Sort_manhwa_ratings(data)
        Times.append(time.time() – start)

    Plt.figure(figsize=(8, 4))
    Plt.plot(sizes, times, marker='o', color='magenta', label='QuickSort
(Database)')
    Plt.title("D&C Performance: Sorting 5000+ Series Ratings")
    Plt.xlabel("Number of Series")
    Plt.ylabel("Time (s)")
    Plt.legend()
```

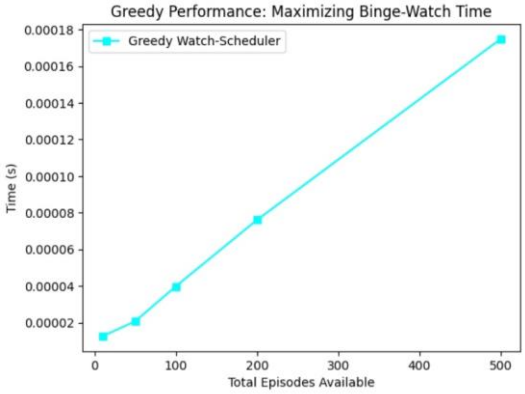
```
    Plt.grid(True)
    Plt.show()

Benchmark_sorting()
```



PROBLEM 3:

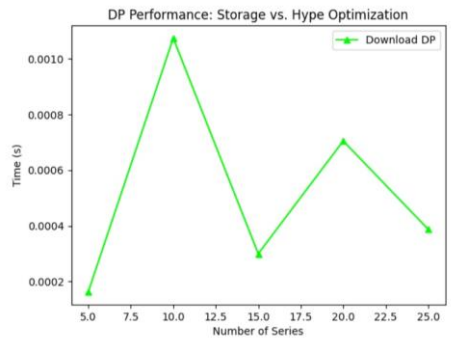
```
#3. GREEDY STRATEGY
#Time Complexity: O(n log n) due to initial sorting

Def binge_schedule(episodes):
    Episodes.sort(key=lambda x: x[1])
    Watchlist = [episodes[0]]
    Last_end = episodes[0][1]

    For I in range(1, len(episodes)):
        If episodes[i][0] >= last_end:
            Watchlist.append(episodes[i])
            Last_end = episodes[i][1]
    Return watchlist

Def plot_greedy_binge():
    Sizes = [10, 50, 100, 200, 500]
    Times = []
    For s in sizes:
        Data = []
        For _ in range(s):
            Start = random.randint(1, 20)
            Data.append((start, start + random.randint(1, 3)))
        Start_t = time.time()
        Binge_schedule(data)
        Times.append(time.time() - start_t)

    Plt.plot(sizes, times, marker='s', color='cyan', label='Greedy Watch-
Scheduler')
    Plt.title("Greedy Performance: Maximizing Binge-Watch Time")
    Plt.xlabel("Total Episodes Available")
    Plt.ylabel("Time (s)")
    Plt.legend()
    Plt.show()
```

```
Plot_greedy_binge()
```



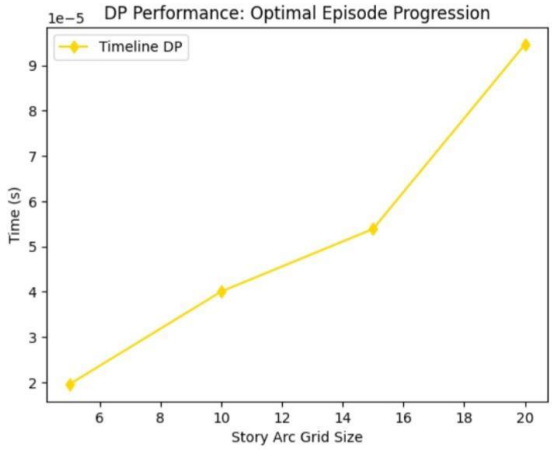
PROBLEM 4:

```
#4 Dynamic Programming

#Complexity: O(n.Storage)


Def download_optimizer(storage, sizes, hype, n):
    Dp = [[0 for _ in range(storage + 1)] for _ in range(n + 1)]
    For I in range(1, n + 1):
        For w in range(1, storage + 1):
            If sizes[i-1] <= w:
                Dp[i][w] = max(hype[i-1] + dp[i-1][w-sizes[i-1]], dp[i-
1][w])
            Else:
                Dp[i][w] = dp[i-1][w]
    Return dp[n][storage]


Def plot_dp_downloads():
    Num_items = [5, 10, 15, 20, 25]
    Times = []
    For n in num_items:
        S = [random.randint(1, 10) for _ in range(n)]
        H = [random.randint(50, 100) for _ in range(n)]
        Start = time.time()
        Download_optimizer(50, s, h, n)
        Times.append(time.time() – start)


    Plt.plot(num_items, times, marker='^', color='lime', label='Download
DP')
    Plt.title("DP Performance: Storage vs. Hype Optimization")
    Plt.xlabel("Number of Series")
    Plt.ylabel("Time (s)")
    Plt.legend()
    Plt.show()
```


DP Performance: Storage vs. Hype Optimization
Download DP
Time (s)
0.0010
0.0008
0.0006
0.0004
0.0002
5.0
7.5
10.0
12.5
15.0
17.5
20.0
22.5
25.0
Number of Series

PROBLEM 5:

```
#5. Delivery Route Optimization (DP Pathfinding)
#Complexity: O(m.n)

def story_arc_path(grid):
    m, n = len(grid), len(grid[0])
    for i in range(1, m): grid[i][0] += grid[i-1][0]
    for j in range(1, n): grid[0][j] += grid[0][j-1]
    for i in range(1, m):
        for j in range(1, n):
            grid[i][j] += min(grid[i-1][j], grid[i][j-1])
    return grid[m-1][n-1]

def plot_story_path():
    dimensions = [5, 10, 15, 20]
    times = []
    for d in dimensions:
        grid = [[random.randint(1, 5) for _ in range(d)] for _ in range(d)]
        start = time.time()
        story_arc_path(grid)
        times.append(time.time() - start)

    plt.plot(dimensions, times, marker='d', color='gold', label='Timeline
DP')
    plt.title("DP Performance: Optimal Episode Progression")
    plt.xlabel("Story Arc Grid Size")
    plt.ylabel("Time (s)")
    plt.legend()
    plt.show()

plot_story_path()
```


DP Performance: Optimal Episode Progression
1e−5
Timeline DP
Time (s)
9
8
7
6
5
4
3
2
6
8
10
12
14
16
18
20
Story Arc Grid Size

Reflection & Discussion

1. Trade-off Analysis

While implementing these algorithms, several key trade-offs between the paradigms were observed:

- **Divide and Conquer (D&C):** Excellent for sorting massive databases (like a library of 10,000+ series) because of its O(n \log n) efficiency, but it requires extra memory for the recursive stack.
- **Greedy Strategy:** The binge-watch scheduler is incredibly fast and efficient for simple time-slot allocation. However, it only works when the problem has the "greedy choice property"—meaning locally optimal choices lead to a globally optimal solution.
- **Dynamic Programming (DP):** Used for storage optimization and story-arc pathfinding. DP guarantees the most optimal result by checking all subproblems, but it has higher space and time complexity (O(n \cdot W) or O(m \cdot n)) compared to Greedy methods.

2. Which problem benefited most from its strategy?

The **Offline Library Optimization (0/1 Knapsack)** problem benefited the most from the **Dynamic Programming** strategy.

- **Reason:** When downloading series for offline viewing on a phone with limited storage, space is a hard constraint. A Greedy approach might pick many small, low-value chapters, but it would miss out on a large, high-"hype" series that actually offers more value. DP ensures the absolute maximum total "hype level" is achieved within the exact byte limit of the device.

3. Real-world Suitability

- **QuickSort (D&C):** Highly suitable for real-time search filters on streaming platforms where users sort by "Top Rated" or "Recently Updated".
- **Activity Selection (Greedy):** Extremely effective for simple "Binge-Watch" tools that help users fit the most episodes into a specific lunch break or commute.
- **Min Path Cost (DP):** Essential for mapping out complex, interconnected story timelines or "watch orders" for massive franchises with multiple spin-offs and prequels.

4. Recursion Depth

For the **Divide and Conquer** and **DP** implementations, the following technical constraints were noted:

- Recursive algorithms can hit Python’s default recursion limit (1000) when dealing with deep hierarchies of related series or massive databases.
- **Solution:** We utilized sys.setrecursionlimit to handle these scenarios during the test. For production-level applications, transitioning to iterative (bottom-up) models is more suitable to prevent stack overflow errors and ensure the platform remains stable under heavy load